

## Basics

1. What is JavaScript, and how is it different from Java?
2. What is the difference between `var`, `let`, and `const`?
3. What are primitive and non-primitive data types?
4. What is the difference between `null` and `undefined`?
5. What is the difference between `==` and `===`?
6. What are truthy and falsy values?
7. What is type coercion?
8. What is `NaN`, and how can you check for it?
9. What is the difference between a function declaration and a function expression?
10. What is an arrow function?
11. What are template literals?
12. What are default parameters?
13. What are rest and spread operators?
14. What is destructuring?
15. What is strict mode?

## Scope and functions

16. What is scope in JavaScript?
17. What is the difference between global, function, and block scope?
18. What is lexical scope?
19. What is a closure? Give a practical example.
20. What is hoisting?
21. How are `var`, `let`, and `const` hoisted differently?
22. What is the temporal dead zone?
23. What is a callback function?
24. What is a higher-order function?
25. What is a pure function?
26. What is an IIFE?
27. What is function currying?
28. What is recursion?
29. What does `this` represent?
30. How does `this` behave inside an arrow function?
31. Explain `call()`, `apply()`, and `bind()`.

## Arrays and objects

32. What is the difference between `map()`, `filter()`, and `reduce()`?
33. What is the difference between `forEach()` and `map()`?
34. What is the difference between `find()` and `filter()`?
35. What is the difference between `slice()` and `splice()`?
36. Which array methods mutate the original array?
37. What is object destructuring?
38. What is the difference between shallow copy and deep copy?
39. How can you clone an object?
40. What are object references?
41. What is immutability, and why is it useful in React?

- 42. What is optional chaining?
- 43. What is nullish coalescing?
- 44. What is the difference between `Object.keys()`, `Object.values()`, and `Object.entries()`?

## Asynchronous JavaScript

- 45. What is synchronous JavaScript?
- 46. What is asynchronous JavaScript?
- 47. What is the call stack?
- 48. What are Browser Web APIs?
- 49. What is the event loop?
- 50. What is the difference between microtasks and macrotasks?
- 51. What is callback hell?
- 52. What is a Promise?
- 53. What are the states of a Promise?
- 54. What is Promise chaining?
- 55. What is the difference between `.then()` and `await`?
- 56. Explain `Promise.all()`.
- 57. Explain `Promise.allSettled()`.
- 58. Explain `Promise.race()`.
- 59. Explain `Promise.any()`.
- 60. How do you handle errors with `async/await`?
- 61. What happens when one Promise fails inside `Promise.all()`?

## Advanced and practical concepts

- 62. What is prototype inheritance?
- 63. What is the prototype chain?
- 64. What is the difference between `prototype` and `__proto__`?
- 65. What are iterators?
- 66. What are generators?
- 67. What is debouncing?
- 68. What is throttling?
- 69. What is event bubbling?
- 70. What is event capturing?
- 71. What is event delegation?
- 72. What is garbage collection?
- 73. What are JavaScript modules?
- 74. What is the difference between named and default exports?
- 75. What is a dynamic import?
- 76. What are `Set` and `Map`?
- 77. How is a `Map` different from a normal object?
- 78. What is memoization?
- 79. What is local storage?
- 80. What is the difference between local storage, session storage, and cookies?

## Output-based interview questions

### Question 1: Hoisting

```
console.log(a);  
var a = 10;
```

What is the output, and why?

---

### Question 2: Temporal dead zone

```
console.log(a);  
let a = 10;
```

What happens when this code executes?

---

### Question 3: Type coercion

```
console.log(1 + "2");  
console.log("5" - 2);  
console.log(true + 1);
```

Find the outputs and explain them.

---

### Question 4: Equality

```
console.log(0 == false);  
console.log(0 === false);  
console.log(null == undefined);  
console.log(null === undefined);
```

Find the outputs.

---

### Question 5: Object reference

```
const user1 = {  
  name: "Meghana"  
};  
  
const user2 = user1;  
  
user2.name = "Deepak";  
  
console.log(user1.name);
```

What is the output, and why?

---

### Question 6: Shallow copy

```
const original = {  
  name: "Meghana",  
  address: {  
    city: "Bangalore"  
  }  
};  
  
const copied = { ...original };  
  
copied.address.city = "Mysore";  
  
console.log(original.address.city);
```

What is the output?

---

### Question 7: Closure

```
function outer() {  
  let count = 0;  
  
  return function () {  
    count++;  
    console.log(count);  
  };  
}  
  
const counter = outer();  
  
counter();  
counter();  
counter();
```

What is the output, and how does closure work here?

---

### Question 8: `setTimeout()`

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 0);  
  
console.log("C");
```

What is the output order?

---

### Question 9: Promise and timer

```
console.log("Start");

setTimeout(() => {
  console.log("Timer");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

What is the output order? Explain microtasks and macrotasks.

---

### Question 10: var inside a loop

```
for (var i = 1; i <= 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
```

What is the output? How would you fix it?

---

### Question 11: this

```
const user = {
  name: "Meghana",

  regularFunction() {
    console.log(this.name);
  },

  arrowFunction: () => {
    console.log(this.name);
  }
};

user.regularFunction();
user.arrowFunction();
```

Why do these methods behave differently?

---

### Question 12: Array methods

```
const numbers = [1, 2, 3, 4, 5];
```

```
const result = numbers
  .filter(number => number % 2 === 0)
  .map(number => number * 10);

console.log(result);
```

What is the output?

---

### Question 13: `map()` with `parseInt()`

```
console.log(["1", "2", "3"].map(parseInt));
```

What is the output, and why?

---

### Question 14: Promise error handling

```
Promise.resolve(10)
  .then(value => value * 2)
  .then(() => {
    throw new Error("Something went wrong");
  })
  .catch(() => 50)
  .then(value => console.log(value));
```

What is the final output?

---

### Question 15: Generator

```
function* generateNumbers() {
  yield 1;
  yield 2;
  return 3;
}

const generator = generateNumbers();

console.log(generator.next());
console.log(generator.next());
console.log(generator.next());
console.log(generator.next());
```

Find the output of every `next()` call.

## Logical programming questions

### Easy level

1. Reverse a string without using `reverse()`.

Input: "hello"  
Output: "olleh"

2. Check whether a string is a palindrome.

Input: "madam"  
Output: true

3. Find the largest number in an array.

Input: [10, 45, 2, 99, 30]  
Output: 99

4. Calculate the sum of all numbers in an array.

Input: [10, 20, 30]  
Output: 60

5. Count the vowels in a string.

Input: "javascript"  
Output: 3

6. Remove duplicate values from an array.

Input: [1, 2, 2, 3, 1, 4]  
Output: [1, 2, 3, 4]

7. Find the factorial of a number.

Input: 5  
Output: 120

8. Print the Fibonacci series up to `n` terms.

Input: 6  
Output: [0, 1, 1, 2, 3, 5]

## Medium-level questions

9. Find the second-largest number without sorting the array.

Input: [10, 5, 20, 8, 20, 15]  
Output: 15

10. Count the frequency of every character.

Input: "banana"  
Output: {  
 b: 1,  
 a: 3,

```
n: 2
}
```

11. Find the first non-repeating character.

Input: "aabbcddee"  
Output: "c"

12. Find the missing number.

Input: [1, 2, 3, 5, 6]  
Output: 4

13. Move all zeros to the end without changing the order of other values.

Input: [0, 1, 0, 3, 12]  
Output: [1, 3, 12, 0, 0]

14. Check whether two strings are anagrams.

Input: "listen", "silent"  
Output: true

15. Find common values between two arrays.

Input: [1, 2, 3, 4], [3, 4, 5, 6]  
Output: [3, 4]

16. Flatten a nested array.

Input: [1, [2, [3, 4]], 5]  
Output: [1, 2, 3, 4, 5]

17. Group objects based on a property.

```
Input: [
  { name: "A", department: "IT" },
  { name: "B", department: "HR" },
  { name: "C", department: "IT" }
]
```

Expected output:

```
{
  IT: [
    { name: "A", department: "IT" },
    { name: "C", department: "IT" }
  ],
  HR: [
    { name: "B", department: "HR" }
  ]
}
```

18. Sort an array of objects based on age.

```
Input: [
  { name: "A", age: 30 },
  { name: "B", age: 20 },
  { name: "C", age: 25 }
]
```

19. Convert an array into an object using `reduce()`.

```
Input: [
  { id: 1, name: "A" },
  { id: 2, name: "B" }
]
```

Expected output:

```
{
  1: { id: 1, name: "A" },
  2: { id: 2, name: "B" }
}
```

20. Create your own implementation of:

`Array.prototype.map`

21. Implement a debounce function.
22. Implement a throttle function.
23. Write a function that retries a failed API request three times.
24. Write a function that executes multiple Promises in sequence.
25. Implement a counter using closure with:

```
increment()
decrement()
getValue()
reset()
```

## Scenario-based interview questions

1. A search API is called for every keyboard entry. How would you optimize it?
2. A scroll event makes the application slow. What solution would you use?
3. Multiple independent APIs must complete before rendering the page. Which Promise method would you use?
4. You need results from every API even when some APIs fail. Which Promise method is suitable?
5. What would you do if an API response arrived after the user moved to another page?
6. How would you identify and fix a JavaScript memory leak?
7. When would you choose `Map` instead of a normal object?
8. When would you use `Promise.allSettled()` instead of `Promise.all()`?